

✓ InfiGen Assignment A — GenAI Analyst

Data Analysis and NLP

Candidate ID: B211623

This notebook loads the Media and Twitter Excel files, standardizes and merges them, creates `Combined`, performs GenAI entity/topic/sentiment analysis, and exports `predictions_B211623.csv`.

API fix: This version uses the Gemini **Interactions API** with `gemini-3.6-flash`. It does **not** use `generate_content()` or `gemini-2.5-flash`.

✓ 1. Install packages

```
# Install only the packages needed by this notebook.
# Do not upgrade pandas/google-auth in Colab because Colab pins compatible versions.
!pip -q install "google-genai>=2.3.0" openpyxl pydantic tqdm
print("Required packages installed.")
```

Required packages installed.

```
# 2. Imports
import os
import time
import random
import json
from typing import List, Literal

import pandas as pd
from pydantic import BaseModel, Field
from tqdm.auto import tqdm

print("Imports successful.")
```

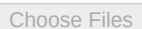
Imports successful.

✓ 3. Upload the two Excel files

Required filenames:

- `Media & Research Articles data.xlsx`
- `Twitter Posts Data.xlsx`

```
# Upload files when running in Google Colab.
try:
    from google.colab import files
    uploaded = files.upload()
    print("Uploaded:", list(uploaded.keys()))
except ImportError:
    print("Not running in Google Colab. Put the Excel files in the current folder.")
```

 2 files

Media & Research Articles data.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 106480 bytes, last modified: 8/19/2026 - 100% done

Twitter Posts Data.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 24838 bytes, last modified: 8/19/2026 - 100% done

Saving Media & Research Articles data.xlsx to Media & Research Articles data.xlsx

Saving Twitter Posts Data.xlsx to Twitter Posts Data.xlsx

Uploaded: ['Media & Research Articles data.xlsx', 'Twitter Posts Data.xlsx']

```
# 4. File checks
MEDIA_FILE = "Media & Research Articles data.xlsx"
TWITTER_FILE = "Twitter Posts Data.xlsx"

if not os.path.exists(MEDIA_FILE):
    raise FileNotFoundError(f"Missing: {MEDIA_FILE}")

if not os.path.exists(TWITTER_FILE):
    raise FileNotFoundError(f"Missing: {TWITTER_FILE}")

print("Both input files found.")
```

Both input files found.

✓ 4. Load and inspect data

```
media_raw = pd.read_excel(MEDIA_FILE)
twitter_raw = pd.read_excel(TWITTER_FILE)

print("Media shape:", media_raw.shape)
print("Twitter shape:", twitter_raw.shape)

print("\nMedia columns:")
print(media_raw.columns.tolist())

print("\nTwitter columns:")
print(twitter_raw.columns.tolist())

display(media_raw.head())
display(twitter_raw.head())
```

```
Media shape: (50, 7)
Twitter shape: (50, 8)

Media columns:
['unique_id', 'Article title', 'Content', 'Source', 'Source Type', 'Article link', 'Published']

Twitter columns:
['unique_id', 'HCP Handle', 'Posts', 'replied_to_tweet', 'Source', 'Source Type', 'Post link', 'Published']
```

	unique_id	Article title	Content	Source	Source Type	Article link
0	Ascopost_51_Q2_2024	USPSTF Issues Final Recommendation Statement ...	On April 30, 2024, the U.S. Preventive Service...	Ascopost	Media Sources	https://ascopc
1	Ascopost_267_Q2_2024	Dennis J. Slamon, MD, PhD, Awarded 2024 Szent...	The National Foundation for Cancer Research (N...	Ascopost	Media Sources	https://ascopost.com/r
2	CancerNetwork_90_Q2_2024	Nivolumab Combo Significantly Boosts Survival ...	Findings from the CheckMate-901 study support ...	CancerNetwork	Media Sources	https://www.cancernetwor
3	MedScape_50_Q2_2024	No Improvement in OS With Atezolizumab in Earl...	Adding atezolizumab to chemotherapy in patient...	MedScape	Media Sources	https://www.medsca
4	MedScape_98_Q2_2024	ADCs for Breast Cancer: Clear Benefits, Manage...	Antibody drug conjugates (ADCs) — a class of t...	MedScape	Media Sources	https://www.medscape

	unique_id	HCP Handle	Posts	replied_to_tweet	Source	Source Type	Post link
0	Twitter_2185_Q2_2024	shilpaonc	@TiansterZhang @ASCO @OncoAlert @DrChoueiri @E...	@shilpaonc up on B1 @ASCO main stage presentin...	Twitter	Social Media	https://twit
1	Twitter_2300_Q2_2024	shilpaonc	@AndreaNecchi @ASCO @Anto_cigliola @MyUniSR @S...	Prepping the next @ASCO presentation by the on...	Twitter	Social Media	https://twit
2	Twitter_2352_Q2_2024	shilpaonc	@CParkMD @ASCO @BourlonMaite @SuzanneColeMD @t...	@shilpaonc @ASCO @BourlonMaite @SuzanneColeMD ...	Twitter	Social Media	https://twit
3	Twitter_2968_Q2_2024	DrLisaCarey	@hoperugo @OncoAlert @KalinskyKevin You have m...	Long awaited post monarch. Abema post mostly p...	Twitter	Social Media	https://twitt
4	Twitter_3680_Q2_2024	ErikaHamilton9	@itsnot_pink @SarahCannonDocs @DavidRSpigel @V...	@ErikaHamilton9 @SarahCannonDocs @DavidRSpigel...	Twitter	Social Media	https://twi

```

print("Media missing values:")
display(media_raw.isna().sum().to_frame("missing"))

print("\nTwitter missing values:")
display(twitter_raw.isna().sum().to_frame("missing"))

print("\nDuplicate Media IDs:", media_raw["unique_id"].duplicated().sum())
print("Duplicate Twitter IDs:", twitter_raw["unique_id"].duplicated().sum())

```

Media missing values:

	missing	
unique_id	0	
Article title	0	
Content	0	
Source	0	
Source Type	0	
Article link	0	
Published date	0	

Twitter missing values:

	missing
unique_id	0
HCP Handle	0
Posts	0
replied_to_tweet	0
Source	0
Source Type	0
Post link	0
Published date	0

Duplicate Media IDs: 0
Duplicate Twitter IDs: 0

✓ 5. Standardize and merge

```

# Media: Article title -> Title, Content -> Body
media = pd.DataFrame({
    "unique_id": media_raw["unique_id"].astype(str),
    "Title": media_raw["Article title"].fillna("").astype(str).str.strip(),
    "Body": media_raw["Content"].fillna("").astype(str).str.strip(),
    "Source": "Media"
})

# Twitter: Posts -> Body, no title
twitter = pd.DataFrame({
    "unique_id": twitter_raw["unique_id"].astype(str),
    "Title": "",
    "Body": twitter_raw["Posts"].fillna("").astype(str).str.strip(),
    "Source": "Twitter"
})

```

```

})

# Combine and HARD-DEDUPE by unique_id + Source.
# This protects against accidental duplicate execution of this cell.
combined_df = pd.concat([media, twitter], ignore_index=True)
combined_df = (
    combined_df
    .drop_duplicates(subset=["unique_id", "Source"], keep="first")
    .reset_index(drop=True)
)

combined_df["Combined"] = (
    combined_df["Title"].fillna("") + "\n" +
    combined_df["Body"].fillna("")
).str.strip()

print("Media rows:", len(media))
print("Twitter rows:", len(twitter))
print("Combined unique rows:", len(combined_df))
assert len(media) == 50
assert len(twitter) == 50
assert len(combined_df) == 100

```

```

Media rows: 50
Twitter rows: 50
Combined unique rows: 100

```

✓ 6. Gemini setup — Interactions API

```

from google import genai

# Get the API key from Colab Secrets or an environment variable.
try:
    from google.colab import userdata
    GEMINI_API_KEY = userdata.get("GEMINI_API_KEY")
except Exception:
    GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")

if not GEMINI_API_KEY:
    raise ValueError(
        "GEMINI_API_KEY not found. In Colab, open Secrets, create "
        "GEMINI_API_KEY, paste your API key, and enable notebook access."
    )

client = genai.Client(api_key=GEMINI_API_KEY)

# IMPORTANT: Do not change this back to gemini-2.5-flash.
MODEL = "gemini-3.6-flash"

print("Gemini client ready.")
print("Model:", MODEL)
print("API: Interactions API")

```

```

Gemini client ready.
Model: gemini-3.6-flash
API: Interactions API

```

7. Structured NLP schema

```

Topic = Literal[
    "Efficacy-General",
    "Progression Free Survival (PFS)",
    "Overall Survival (OS)",
    "Safety-General",
    "Safety-Side Effects",
    "General Opinion",
    "Others",
]

Sentiment = Literal["Positive", "Negative", "Neutral"]

class NLPResult(BaseModel):
    drugs: List[str] = Field(default_factory=list)
    diseases: List[str] = Field(default_factory=list)
    study_names: List[str] = Field(default_factory=list)
    topic: Topic
    sentiment: Sentiment

class BatchItem(BaseModel):
    record_index: int
    drugs: List[str] = Field(default_factory=list)
    diseases: List[str] = Field(default_factory=list)
    study_names: List[str] = Field(default_factory=list)
    topic: Topic
    sentiment: Sentiment

class BatchNLPResult(BaseModel):
    results: List[BatchItem] = Field(default_factory=list)

JSON_SCHEMA = BatchNLPResult.model_json_schema()

print(json.dumps(JSON_SCHEMA, indent=2))

```

```

{
  "$defs": {
    "BatchItem": {
      "properties": {
        "record_index": {
          "title": "Record Index",
          "type": "integer"
        },
        "drugs": {
          "items": {
            "type": "string"
          },
          "title": "Drugs",
          "type": "array"
        },
        "diseases": {
          "items": {
            "type": "string"
          },
          "title": "Diseases",
          "type": "array"
        },
        "study_names": {
          "items": {

```

```

        "type": "string"
    },
    "title": "Study Names",
    "type": "array"
},
"topic": {
    "enum": [
        "Efficacy-General",
        "Progression Free Survival (PFS)",
        "Overall Survival (OS)",
        "Safety-General",
        "Safety-Side Effects",
        "General Opinion",
        "Others"
    ],
    "title": "Topic",
    "type": "string"
},
"sentiment": {
    "enum": [
        "Positive",
        "Negative",
        "Neutral"
    ],
    "title": "Sentiment",
    "type": "string"
}
},
"required": [
    "record_index",
    "topic",
    "sentiment"
],
"title": "BatchItem"

```

8. NLP instructions

SYSTEM_INSTRUCTION = ""

You are a pharmaceutical and medical NLP analyst.

You will receive multiple media articles or Twitter posts. Each item has a numeric record_index. Analyze each item independently.

For EACH record, extract explicitly mentioned:

1. Drugs: drugs, medicines, therapies, treatments, compounds, drug candidates.
2. Diseases: diseases, cancers, disorders, conditions, indications.
3. Study Names: clinical trials, research studies, named studies, trial identifiers.

Never invent or infer entities. If an entity is not explicitly supported by that record's text return an empty list.

Choose exactly ONE primary topic for each record:

- Efficacy-General: general treatment/drug effectiveness or efficacy.
- Progression Free Survival (PFS): specifically PFS or progression-free survival.
- Overall Survival (OS): specifically OS, overall survival, mortality, or survival duration.
- Safety-General: general safety, tolerability, or risk.
- Safety-Side Effects: specific adverse events, side effects, or toxicities.
- General Opinion: opinions, reactions, praise, criticism, recommendations, perceptions.
- Others: content that does not fit the above.

Choose exactly ONE sentiment for each record:

- Positive
- Negative
- Neutral

Return exactly one result for every supplied record_index, preserving the same record_index value.

```
Return only JSON matching the supplied schema.
"""
```

✓ 9. Gemini analysis function

```
class QuotaExceededError(RuntimeError):
    """Raised when Gemini returns HTTP 429/quota exhaustion."""
    pass

def analyze_batch(records, retries=2):
    """Analyze a small batch of records in one Gemini Interactions API call."""
    if not records:
        return {}

    batch_input_parts = []
    for record_index, record in records:
        text = str(record["Combined"]).strip()
        batch_input_parts.append(
            f"RECORD_INDEX: {record_index}\nTEXT:\n{text}"
        )

    batch_input = (
        "Analyze every record below independently. "
        "Return one JSON result per RECORD_INDEX.\n\n"
        + "\n\n--- NEXT RECORD ---\n\n".join(batch_input_parts)
    )

    last_error = None

    for attempt in range(1, retries + 1):
        try:
            interaction = client.interactions.create(
                model=MODEL,
                system_instruction=SYSTEM_INSTRUCTION,
                input=batch_input,
                response_format={
                    "type": "text",
                    "mime_type": "application/json",
                    "schema": JSON_SCHEMA,
                },
            )

            raw = interaction.output_text
            if not raw:
                raise ValueError("Gemini returned empty output.")

            parsed = BatchNLPResult.model_validate_json(raw)

            result_map = {item.record_index: item for item in parsed.results}
            expected = {idx for idx, _ in records}

            if set(result_map) != expected:
                missing = expected - set(result_map)
                extra = set(result_map) - expected
                raise ValueError(
                    f"Batch alignment error. Missing={sorted(missing)}, "
                    f"Extra={sorted(extra)}"
                )

            return result_map
```

```

except Exception as exc:
    last_error = exc
    message = str(exc)

    # Quota/rate-limit errors should NOT be retried repeatedly.
    if "429" in message or "RateLimitError" in type(exc).__name__:
        raise QuotaExceededError(
            "Gemini quota/rate limit reached. "
            "Stop this run and resume later; completed checkpoint rows are safe."
        ) from exc

    # A model-not-found error is permanent.
    if "404" in message and (
        "NOT_FOUND" in message
        or "no longer available" in message.lower()
        or "not found" in message.lower()
    ):
        raise RuntimeError(
            f"Model '{MODEL}' is unavailable for this API key. "
            "Use a currently available Interactions API model."
        ) from exc

    print(
        f"Batch attempt {attempt}/{retries} failed: "
        f"{type(exc).__name__}: {exc}"
    )

    if attempt < retries:
        wait = min(20, (2 ** (attempt - 1)) * 2 + random.random())
        print(f"Retrying batch in {wait:.1f} seconds...")
        time.sleep(wait)

    raise RuntimeError(
        f"Gemini batch failed after {retries} attempts."
    ) from last_error

```

✓ 10. Test Gemini before processing the full dataset

```

# Optional API smoke test.
# Set RUN_API_TEST = True if you want to test the connection manually.
# Keeping it False preserves one free-tier request for the actual assignment run.

RUN_API_TEST = False

if RUN_API_TEST:
    sample_records = [
        0,
        {
            "Combined": """USPSTF Issues Final Recommendation Statement on Screening for Breast Cancer

On April 30, 2024, the U.S. Preventive Services Task Force published
a final recommendation statement on screening for breast cancer."""
        }
    ]

    print("=" * 70)
    print("TESTING GEMINI")
    print("=" * 70)
    test_map = analyze_batch(sample_records)
    print(BatchItem.model_validate(test_map[0]).model_dump_json(indent=2))
    print("\nAPI TEST PASSED.")

```

```

else:
    print("API smoke test skipped to conserve quota. The batch-processing cell will use Gemini

API smoke test skipped to conserve quota. The batch-processing cell will use Gemini.

```

✓ 11. Batched NLP processing

To reduce API usage, records are processed in small batches instead of one API request per record. A checkpoint is written after every successful batch.

Recommended batch size: 8 records. With 100 records, this requires about 13 Gemini requests (plus any optional test request).

Files:

- `predictions_B211623_checkpoint.csv`
- `predictions_B211623.csv`
- `failed_records_B211623.csv`

If Colab disconnects or Gemini quota is reached, rerun the notebook later. Already-completed checkpoint records will be skipped.

Important: If you see a Gemini 429 quota error, do not repeatedly rerun the API call immediately. The notebook stops cleanly and preserves the checkpoint. Wait for the quota window to reset and then rerun the batch-processing cell.

```

CANDIDATE_ID = "B211623"

CHECKPOINT_FILE = f"predictions_{CANDIDATE_ID}_checkpoint.csv"
FINAL_FILE = f"predictions_{CANDIDATE_ID}.csv"
FAILED_FILE = f"failed_records_{CANDIDATE_ID}.csv"

OUTPUT_COLUMNS = [
    "unique_id",
    "Source",
    "Drugs",
    "Diseases",
    "Study_Names",
    "Topic",
    "Sentiment",
]

# The source dataset must contain exactly 100 unique records.
source_keys = {
    (str(row["unique_id"]), str(row["Source"]))
    for _, row in combined_df.iterrows()
}
assert len(source_keys) == 100, f"Expected 100 source keys, found {len(source_keys)}"

if os.path.exists(CHECKPOINT_FILE):
    checkpoint_df = pd.read_csv(CHECKPOINT_FILE, dtype=str).fillna("")
    checkpoint_df = checkpoint_df[OUTPUT_COLUMNS].copy()

    # Remove accidental duplicate checkpoint rows.
    checkpoint_df = (
        checkpoint_df
        .drop_duplicates(subset=["unique_id", "Source"], keep="last")
        .reset_index(drop=True)
    )

```

```

# Keep only records that actually exist in the current 100-row source dataset.
checkpoint_df = checkpoint_df[
    checkpoint_df.apply(
        lambda r: (str(r["unique_id"]), str(r["Source"])) in source_keys,
        axis=1
    )
].reset_index(drop=True)

completed_keys = set(
    zip(
        checkpoint_df["unique_id"].astype(str),
        checkpoint_df["Source"].astype(str),
    )
)
successful_rows = checkpoint_df.to_dict("records")

# Immediately rewrite a clean checkpoint.
checkpoint_df.to_csv(CHECKPOINT_FILE, index=False)

print("Checkpoint found.")
print("Raw/duplicate rows removed where necessary.")
print("Clean completed predictions:", len(successful_rows))
else:
    checkpoint_df = pd.DataFrame(columns=OUTPUT_COLUMNS)
    completed_keys = set()
    successful_rows = []
    print("No checkpoint found. Starting from the beginning.")

```

No checkpoint found. Starting from the beginning.

```

def result_to_row(record, result):
    return {
        "unique_id": str(record["unique_id"]),
        "Source": str(record["Source"]),
        "Drugs": "; ".join(result.drugs),
        "Diseases": "; ".join(result.diseases),
        "Study_Names": "; ".join(result.study_names),
        "Topic": result.topic,
        "Sentiment": result.sentiment,
    }

remaining = [
    (idx, row)
    for idx, row in combined_df.iterrows()
    if (str(row["unique_id"]), str(row["Source"])) not in completed_keys
]

print("Total source records:", len(combined_df))
print("Already completed (unique):", len(completed_keys))
print("Remaining:", len(remaining))
assert len(completed_keys) + len(remaining) == 100

```

Total source records: 100
 Already completed (unique): 0
 Remaining: 100

```

# Process ONLY missing source records.
# This version prevents the checkpoint counter from ever exceeding 100.

BATCH_SIZE = 8
failed_rows = []
quota_hit = False

```

```

for start in tqdm(
    range(0, len(remaining), BATCH_SIZE),
    desc="Gemini batches"
):
    batch = remaining[start:start + BATCH_SIZE]

    try:
        result_map = analyze_batch(batch)

        for record_index, record in batch:
            item = result_map[record_index]

            result = NLPResult(
                drugs=item.drugs,
                diseases=item.diseases,
                study_names=item.study_names,
                topic=item.topic,
                sentiment=item.sentiment,
            )

            key = (str(record["unique_id"]), str(record["Source"]))
            row = result_to_row(record, result)

            # Replace any accidental duplicate by key.
            successful_rows = [
                r for r in successful_rows
                if (str(r["unique_id"]), str(r["Source"])) != key
            ]
            successful_rows.append(row)
            completed_keys.add(key)

        # Clean + cap checkpoint to the actual 100 source keys.
        checkpoint_out = (
            pd.DataFrame(successful_rows, columns=OUTPUT_COLUMNS)
            .drop_duplicates(subset=["unique_id", "Source"], keep="last")
        )

        checkpoint_out = checkpoint_out[
            checkpoint_out.apply(
                lambda r: (str(r["unique_id"]), str(r["Source"])) in source_keys,
                axis=1
            )
        ].reset_index(drop=True)

        # Never allow more than the 100 source records.
        assert len(checkpoint_out) <= 100

        checkpoint_out.to_csv(CHECKPOINT_FILE, index=False)
        successful_rows = checkpoint_out.to_dict("records")
        completed_keys = set(
            zip(
                checkpoint_out["unique_id"].astype(str),
                checkpoint_out["Source"].astype(str)
            )
        )

        print(f"Saved checkpoint: {len(successful_rows)} unique predictions.")

        if len(successful_rows) == 100:
            print("ALL 100 RECORDS COMPLETED.")
            break

    except QuotaExceededError as exc:
        quota_hit = True

```

```

print("\n" + "=" * 70)
print("GEMINI QUOTA/RATE LIMIT REACHED")
print("=" * 70)
print(str(exc))
print(
    "Completed unique predictions are already saved in "
    f"{CHECKPOINT_FILE}."
)
print("Rerun this cell later; it will skip completed records.")
break

except Exception as exc:
    for record_index, record in batch:
        failed_rows.append({
            "unique_id": str(record["unique_id"]),
            "Source": str(record["Source"]),
            "error_type": type(exc).__name__,
            "error": str(exc),
        })

    print(
        f"Batch starting at position {start} failed: "
        f"{type(exc).__name__}: {exc}"
    )

print("\nBatched processing finished.")
print("Successful unique predictions available:", len(successful_rows))
print("Failed records this run:", len(failed_rows))
print("Quota hit:", quota_hit)

assert len(successful_rows) <= 100

```

Gemini batches: 92%

12/13 [07:53<00:32, 32.04s/it]

Saved checkpoint: 8 unique predictions.
 Saved checkpoint: 16 unique predictions.
 Saved checkpoint: 24 unique predictions.
 Saved checkpoint: 32 unique predictions.
 Saved checkpoint: 40 unique predictions.
 Saved checkpoint: 48 unique predictions.
 Saved checkpoint: 56 unique predictions.
 Saved checkpoint: 64 unique predictions.
 Saved checkpoint: 72 unique predictions.
 Saved checkpoint: 80 unique predictions.
 Saved checkpoint: 88 unique predictions.
 Saved checkpoint: 96 unique predictions.
 Saved checkpoint: 100 unique predictions.
 ALL 100 RECORDS COMPLETED.

Batched processing finished.
 Successful unique predictions available: 100
 Failed records this run: 0
 Quota hit: False

```

# Save failures separately.
if failed_rows:
    failed_df = pd.DataFrame(failed_rows)
    failed_df.to_csv(FAILED_FILE, index=False)
    print("Failed records saved to:", FAILED_FILE)
else:
    failed_df = pd.DataFrame(
        columns=["unique_id", "Source", "error_type", "error"]
    )

```

```
print("No failed records.")
```

No failed records.

✓ 12. Build final predictions CSV

```
final_predictions = (
    pd.DataFrame(successful_rows, columns=OUTPUT_COLUMNS)
    .drop_duplicates(
        subset=["unique_id", "Source"],
        keep="last"
    )
    .reset_index(drop=True)
)

# Keep only valid source records.
final_predictions = final_predictions[
    final_predictions.apply(
        lambda r: (str(r["unique_id"]), str(r["Source"])) in source_keys,
        axis=1
    )
].reset_index(drop=True)

# Preserve original input order.
order_map = {
    (str(row["unique_id"]), str(row["Source"])): i
    for i, (_, row) in enumerate(combined_df.iterrows())
}

final_predictions["_order"] = final_predictions.apply(
    lambda r: order_map.get(
        (str(r["unique_id"]), str(r["Source"]))
    ),
    axis=1,
)

final_predictions = (
    final_predictions
    .sort_values("_order")
    .drop(columns="_order")
    .reset_index(drop=True)
)

final_predictions.to_csv(FINAL_FILE, index=False)

print("Created/updated:", FINAL_FILE)
print("Prediction rows currently available:", len(final_predictions))
display(final_predictions.head(10))

if len(final_predictions) == 100:
    print("\nSUCCESS: Final submission contains exactly 100 unique predictions.")
else:
    print(
        f"\nINCOMPLETE: {100 - len(final_predictions)} records still need predictions."
    )
    print("Do NOT submit this CSV yet. Resume the batch-processing cell after quota resets.")
```

0	Ascopost_51_Q2_2024	Media		breast cancer		Others	Neutral
1	Ascopost_267_Q2_2024	Media	trastuzumab; palbociclib	breast cancer; HER2-positive breast cancer; es...		General Opinion	Positive
2	CancerNetwork_90_Q2_2024	Media	nivolumab; gemcitabine; cisplatin; Opdivo; ave...	advanced urothelial carcinoma; urothelial carc...	CheckMate-901	Overall Survival (OS)	Positive
3	MedScape_50_Q2_2024	Media	atezolizumab; nab-paclitaxel; carboplatin; gem...	triple-negative breast cancer; TNBC; locally a...	IMpassion132; IMpassion130	Overall Survival (OS)	Negative
4	MedScape_98_Q2_2024	Media	datopotamab deruxtecan; Dato-DXd; eribulin mes...	metastatic breast cancer; breast cancer; metas...	TROPION-Breast01; DESTINY-Breast02; DESTINY-Br...	Safety-Side Effects	Positive
5	MedScape_103_Q2_2024	Media	atezolizumab; nab-paclitaxel; carboplatin; gem...	triple-negative breast cancer; TNBC; locally a...	IMpassion132; IMpassion130	Overall Survival (OS)	Negative
6	MedScape_123_Q2_2024	Media		breast cancer		Others	Negative
7	MedScape_218_Q2_2024	Media	GLP-1 agonists	breast cancer; early breast cancer; receptor-p...	RxPONDER; A-BRAVE	Efficacy-General	Positive
8	Onclive_56_Q2_2024	Media	datopotamab deruxtecan; Dato-DXd; platinum-bas...	advanced non-small cell lung cancer; metastati...	TROPION-Lung05; NCT04484142; 2024 ASCO Annual ...	Efficacy-General	Positive
9	Onclive_59_Q2_2024	Media	nivolumab; Opdivo; chemotherapy; ipilimumab; Y...	resectable non-small cell lung cancer; NSCLC; ...	CheckMate 816; NCT02998528; 2024 ASCO Annual M...	Overall Survival (OS)	Positive

✓ 13. Final quality checks

SUCCESS: Final submission contains exactly 100 unique predictions.

```
valid_topics = {  
    "Efficacy-General",  
}
```

```

    "Progression Free Survival (PFS)",
    "Overall Survival (OS)",
    "Safety-General",
    "Safety-Side Effects",
    "General Opinion",
    "Others",
}

valid_sentiments = {"Positive", "Negative", "Neutral"}

assert set(final_predictions["Topic"]).issubset(valid_topics)
assert set(final_predictions["Sentiment"]).issubset(valid_sentiments)
assert set(final_predictions["Source"]).issubset({"Media", "Twitter"})

key_input = set(
    zip(
        combined_df["unique_id"].astype(str),
        combined_df["Source"].astype(str)
    )
)

key_output = set(
    zip(
        final_predictions["unique_id"].astype(str),
        final_predictions["Source"].astype(str)
    )
)

missing = key_input - key_output

print("=" * 70)
print("FINAL QUALITY REPORT")
print("=" * 70)
print("Input records:", len(combined_df))
print("Prediction records:", len(final_predictions))
print("Missing predictions:", len(missing))

if missing:
    print(
        "\nSTATUS: Processing is incomplete. "
        f"{len(missing)} records remain."
    )
    print(
        "After the Gemini quota resets, rerun the batch-processing cell. "
        "The checkpoint will prevent duplicate work."
    )
else:
    print("\nSUCCESS: All 100 records have predictions.")

print("\nTopic distribution:")
display(final_predictions["Topic"].value_counts().to_frame("count"))

print("\nSentiment distribution:")
display(final_predictions["Sentiment"].value_counts().to_frame("count"))

print("\nSource distribution:")
display(final_predictions["Source"].value_counts().to_frame("count"))

```

```
=====
FINAL QUALITY REPORT
=====
```

```
Input records: 100
Prediction records: 100
Missing predictions: 0
```

SUCCESS: All 100 records have predictions.

Topic distribution:

	count 
Topic	
General Opinion	32
Others	24
Efficacy-General	22
Progression Free Survival (PFS)	9
Overall Survival (OS)	7
Safety-Side Effects	3
Safety-General	3

Sentiment distribution:

	count
Sentiment	
Positive	66
Neutral	27
Negative	7

Source distribution:

	count
Source	
Media	50
Twitter	50

```
# 14. Display final sample
display(final_predictions.head(20))
```


	unique_id	Source	Drugs	Diseases	Study_Names	Topic	Sentir
0	Ascopost_51_Q2_2024	Media		breast cancer		Others	Ne
1	Ascopost_267_Q2_2024	Media	trastuzumab; palbociclib	breast cancer; HER2-positive breast cancer; es...		General Opinion	Po:
2	CancerNetwork_90_Q2_2024	Media	nivolumab; gemcitabine; cisplatin; Opdivo; ave...	advanced urothelial carcinoma; urothelial carc...	CheckMate-901	Overall Survival (OS)	Po:
3	MedScape_50_Q2_2024	Media	atezolizumab; nab-paclitaxel; carboplatin; gem...	triple-negative breast cancer; TNBC; locally a...	IMpassion132; IMpassion130	Overall Survival (OS)	Neg
4	MedScape_98_Q2_2024	Media	datopotamab deruxtecan; Dato-DXd; eribulin mes...	metastatic breast cancer; breast cancer; metas...	TROPION-Breast01; DESTINY-Breast02; DESTINY-Br...	Safety-Side Effects	Po:
5	MedScape_103_Q2_2024	Media	atezolizumab; nab-paclitaxel; carboplatin; gem...	triple-negative breast cancer; TNBC; locally a...	IMpassion132; IMpassion130	Overall Survival (OS)	Neg
6	MedScape_123_Q2_2024	Media		breast cancer		Others	Neg
7	MedScape_218_Q2_2024	Media	GLP-1 agonists	breast cancer; early breast cancer; receptor-p...	RxPONDER; A-BRAVE	Efficacy-General	Po:
8	Onclive_56_Q2_2024	Media	datopotamab deruxtecan; Dato-DXd; platinum-bas...	advanced non-small cell lung cancer; metastati...	TROPION-Lung05; NCT04484142; 2024 ASCO Annual ...	Efficacy-General	Po:

15. Download final submission

9	Onclive_59_Q2_2024	Media	nivolumab; Opdivo;	resectable non-small cell lung	CheckMate 816; NCT02998528;	Overall Survival	Po:
<pre>try: from google.colab import files files.download(FINAL_FILE) except ImportError: print("File saved locally as:", FINAL_FILE)</pre>							
11	Onclive_387_Q2_2024	Media	abemaciclib; Verzenio; fulvestrant; Faslodex; ...	breast cancer; hormone receptor-posi...	postMONARCH; NCT05169567; 2024 ASCO Annual Mee...	Progression Free Survival (PFS)	Po: